KAZAKH-BRITISH TECHNICAL UNIVERSITY

School of IT and Engineering

MACHINE LEARNING

LECTURE #6

Number of credits: 3 (2/0/1)
Course code – CSCI3234
MS Teams team code – **y4ntwlz**

Adilet Yerkin, MS in Engineering, Senior Lecturer
a.yerkin@kbtu.kz

Ensemble Learning

BAGGING

BOOSTING

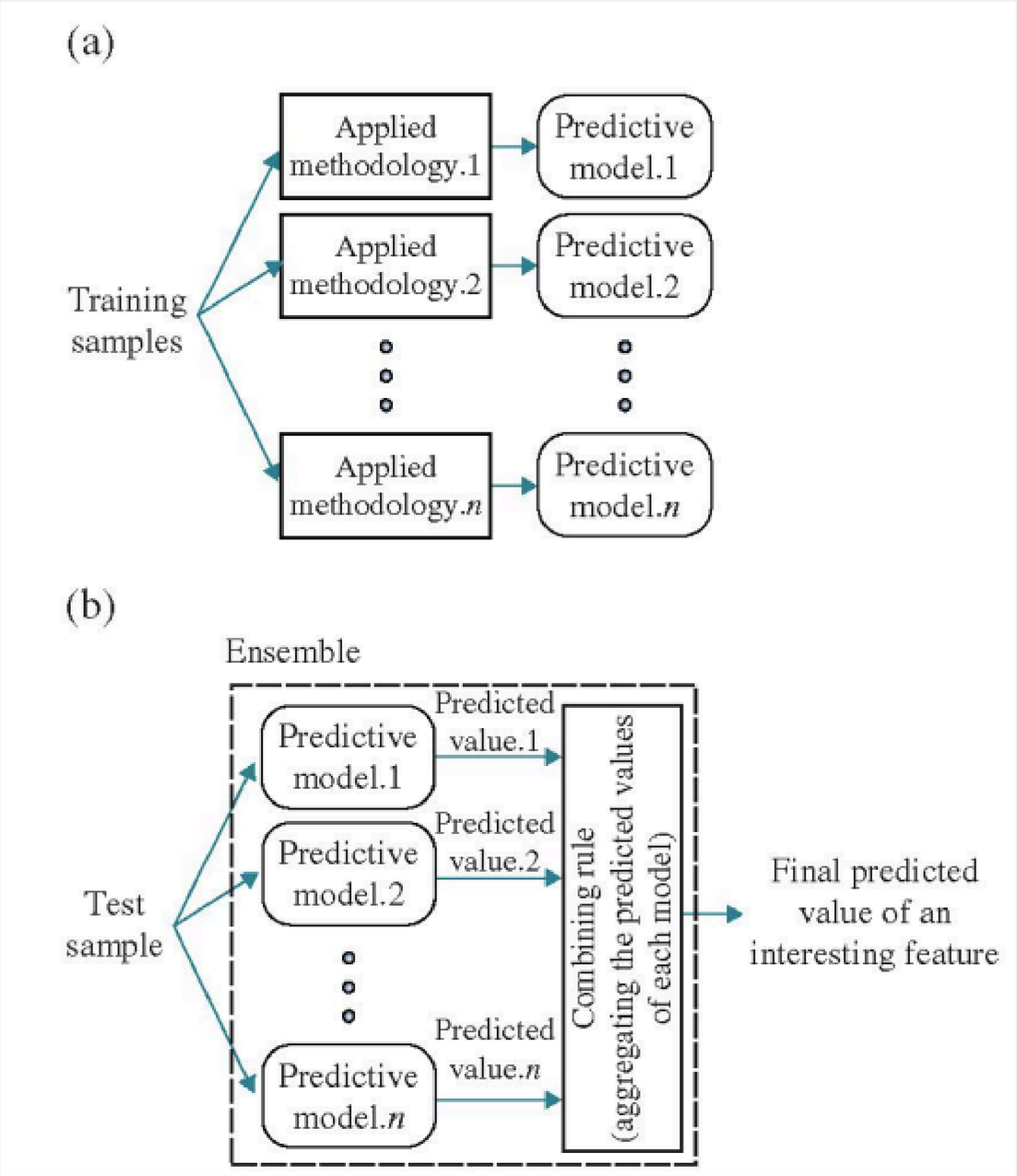
Ensemble methods are motivated by the fact that different classifiers may make different predictions on test instances due to the specific characteristics of the classifier, or their sensitivity to the random artifacts in the training data.

An ensemble method is an approach to increase the prediction accuracy by combining the results from multiple classifiers.

In many machine learning problems, a single model is not enough to achieve reliable performance. Different models make different mistakes.
The idea of ensemble learning is simple: **Instead of trusting one model, we combine multiple models and trust their collective decision.**
This principle is widely used in modern machine learning systems such as fraud detection, recommendation systems, medical diagnosis, and credit scoring.

The ensemble learning methodology

(A) TRAINING PHASE
(B) TESTING PHASE FOR BUILDING AN ENSEMBLE.



Bias and Variance

BIAS

Bias shows how far a model's average predictions are from the real (true) values. It happens when a model makes wrong assumptions about the relationship between input and output.

High-bias models tend to make strong assumptions about the form of the data and cause underfitting. An overly simplistic model tends to have high bias and low variance—a model like this tends to have high training errors and high prediction errors.

High bias → Underfitting
Model cannot learn enough from data
Performs poorly on both training and test data

$$\text{Bias}(x) = \mathbb{E}[\hat{f}(x)] - f(x)$$

Formally, for a given data point x, let f(x) denote the true target function and f^(x) the prediction of the learning algorithm.

Bias and Variance

VARIANCE

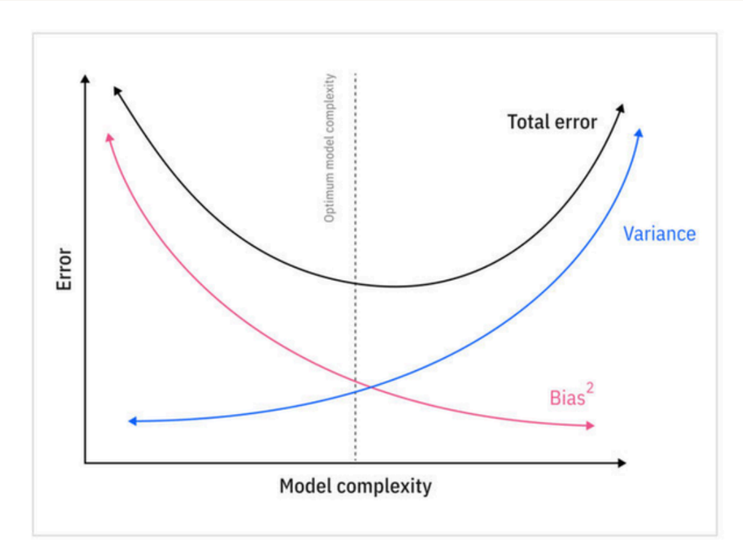
Variance measures how much a model's predictions change with different training datasets. High-variance models are sensitive to noise in the training data and cause overfitting. A model with complex architecture and more parameters tends to have high variance and low bias.

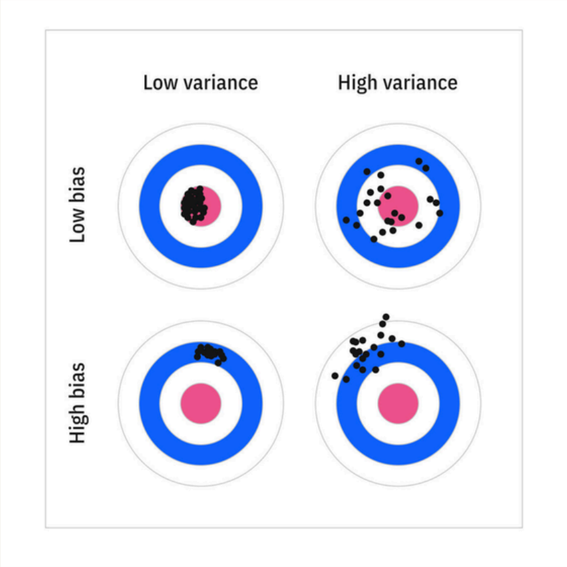
$$\mathrm{Var}(x) = \mathbb{E}[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2]$$

High variance → Overfitting
Model learns noise instead of patterns
Performs well on training data, but poorly on new data

Bias and Variance

TRADEOFF





Model complexity and the number of parameters directly affect bias-variance tradeoff.
As the model becomes more complex and has more parameters, the variability in predicted values in the testing set increases, leading to high variance. However, as the model simplifies and the numbers of parameters decrease, the bias^2 in prediction increases.

Machine learning model, we aim to simultaneously bias and variance to achieve optimum model performance.
This optimization not only generates good results from the training, but also generalize well to unseen testing data

Bias and Variance

TRADEOFF

Hyperparameter tuning and model selection
Model complexity and regularization strength are often controlled through hyperparameters.

Techniques like grid search or random search with cross-validation or Bayesian optimization can help find a model that balances bias and variance on held-out data.

Regularization refers to a set of techniques used to constrain or penalize a model's complexity to improve generalization that is, performance on unseen data. In mathematical terms, regularization modifies the original loss function by adding a penalty term that discourages complexity (usually in the form of large weights or overly flexible models).

Regularization

Regularization is a process of introducing additional information or constraints to a model to prevent it from fitting noise and to improve its generalization ability.

L1 Regularization (Lasso Regression)

$$\text{Loss} = \text{MSE} + \lambda \sum_{i=1}^{n} |w_i|$$

L1 regularization adds the absolute value of each weight to the loss function

Forces some weights to become exactly zero
Performs feature selection automatically
Creates a sparse model (only important features remain)

Regularization

L2 Regularization (Ridge)

$$\text{Loss} = \text{MSE} + \lambda \sum_{i=1}^{n} w_i^2$$

L2 regularization adds the square of each weight to the loss function

Shrinks all weights towards zero, but none become exactly zero
Keeps all features but makes them smaller
Produces a smooth and stable model

λ is the regularization coefficient (controls the strength of the penalty),
A higher λ→ simpler model (model is restricted) (less variance, more bias). Model becomes simpler → possible underfitting
A lower λ → more flexible model (less bias, more variance). Model fits training data closely → possible overfitting

Bagging

1. From the original dataset, create several bootstrap samples (random samples with replacement).
2. Train a separate model (usually the same type, e.g., decision tree) on each sample.
3. Combine predictions by averaging (for regression) or voting (for classification).

Example algorithms:

- Random Forest

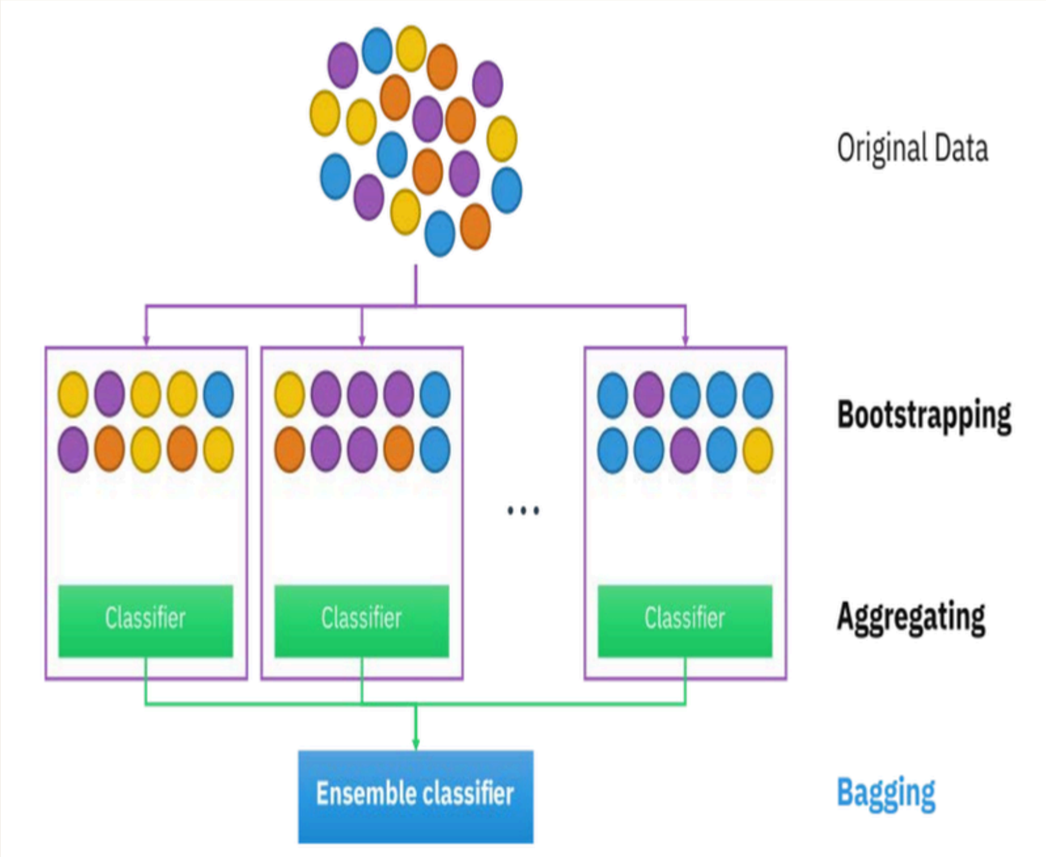
Advantages:

- Reduces overfitting
- Works well with high-variance models (like decision trees)

Disadvantages:

- Does not reduce bias
- Models are trained independently (no learning from mistakes)

Bagging



Idea

Bagging improves both stability and accuracy of machine learning algorithms.

Its main goal is to reduce variance and avoid overfitting.

Some algorithms (like decision trees) are unstable:

a small change in the dataset → completely different model.

Bagging stabilizes them by training many models on slightly different datasets.

Bagging

Bootstrap Sampling

We usually have only one dataset.

To create multiple datasets, bagging uses sampling with replacement.

This means:

- We randomly pick observations from the dataset
- After selecting a sample, we put it back
- The same record can appear multiple times
- Some records may not appear at all

Each new dataset is called a bootstrap sample.

Because each bootstrap is different, each model learns a slightly different pattern.

Training Procedure

1. Generate many bootstrap datasets
2. Train one model on each dataset
3. Combine predictions

Each model is trained independently.

Prediction

Classification

Final prediction = majority vote

Example:

16 models trained

10 predict SPAM

6 predict NOT SPAM

Final answer → SPAM

Regression

Final prediction = average of predictions

Boosting

1. Train a weak model (like a shallow decision tree).
2. Identify the errors made by the model.
3. Train the next model to focus more on those errors.
4. Combine all models in a weighted way models that perform better get higher weight.

Example algorithms:

- AdaBoost (Adaptive Boosting)
- Gradient Boosting
- XGBoost, LightGBM, CatBoost (modern, optimized versions)

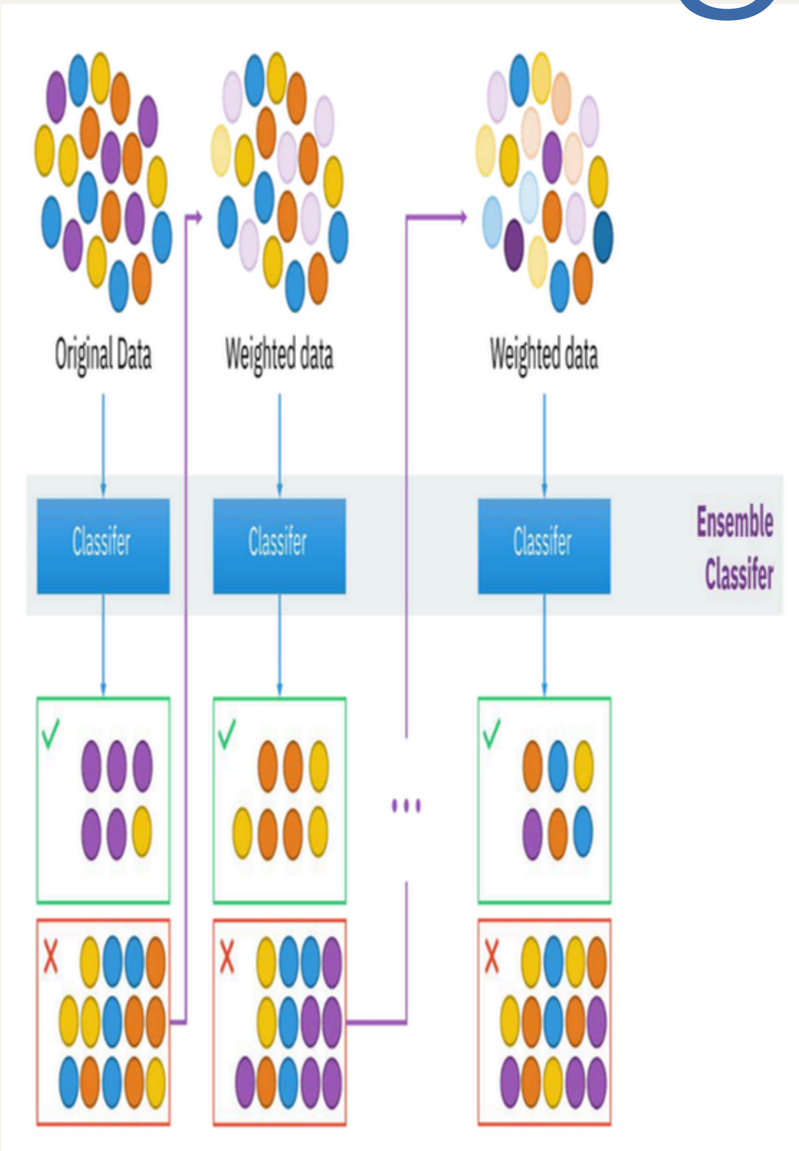
Advantages:

- Produces very accurate models
- Focuses learning on hard-to-predict samples

Disadvantages:

- Sensitive to noise and outliers
- Can overfit if not regularized properly
- Sequential harder to parallelize than bagging

Boosting



Idea

Boosting converts many weak models into one strong model.
Unlike bagging:

- Bagging → independent models
- Boosting → sequential learning

Each new model focuses on correcting mistakes made by previous models.

How Boosting Works

1. Train the first weak model
2. Find incorrectly classified samples
3. Increase their importance (weight)
4. Train the next model focusing on difficult cases
5. Repeat many times

Final model = weighted combination of all models
Models that perform better receive higher influence.

Intuition

Boosting behaves like learning from feedback.
A student solves problems → teacher marks mistakes → student improves → repeat
Each step focuses on harder examples.

Boosting

Gradient Boosting
Modern boosting algorithms use optimization.
Instead of manually reweighting samples, they train each new model to predict residual errors.
So every new model learns what previous models could not explain.

Popular Boosting Algorithms

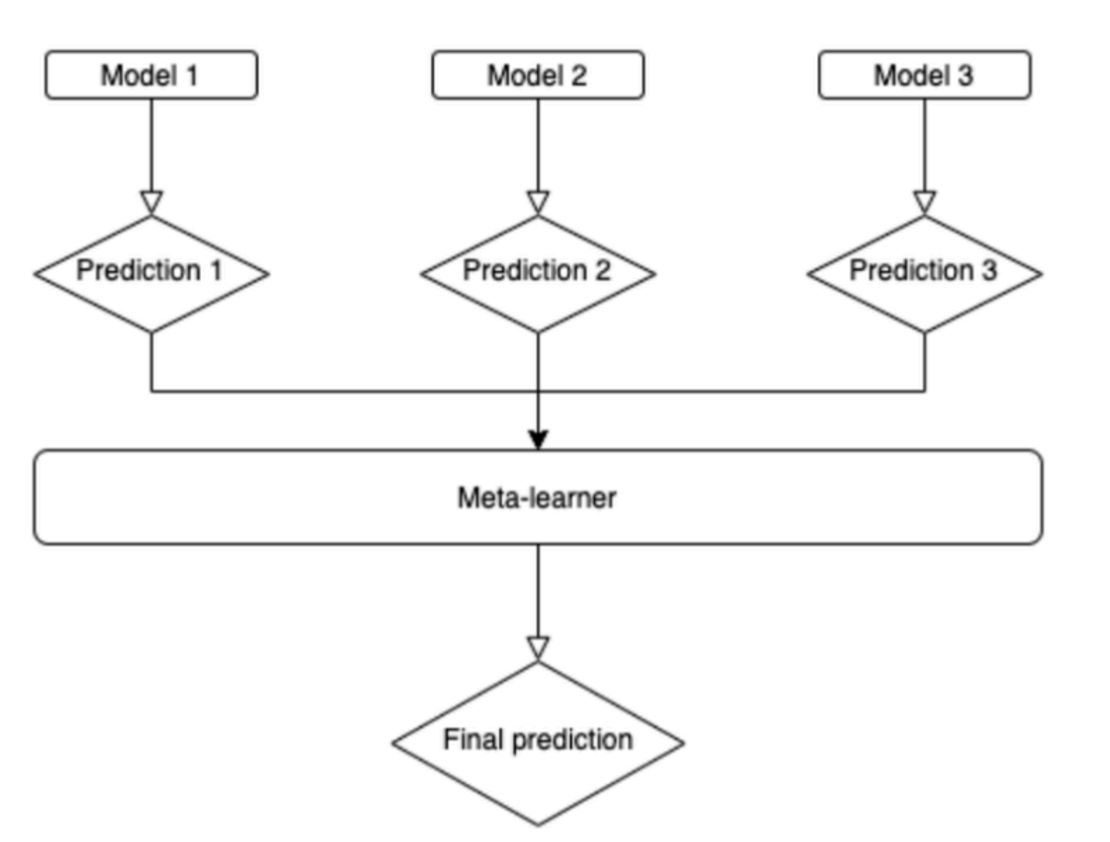
- Gradient Boosting Machine (GBM)
- XGBoost
- LightGBM

LightGBM is popular in industry because it trains faster on large datasets.

Boosting has been used in many real problems including classification, ranking systems, and scientific discovery (e.g., particle physics experiments).

Important Note
Boosting can overfit noisy data. Because it tries to fix every mistake — including errors caused by noise.

Stacking



Idea

Stacking trains multiple models and then trains another model to combine them.
Instead of voting or averaging, we learn how to combine predictions.
The combining model is called a **meta-learner.**

Process

1. Train several base models (different algorithms)
2. Each model predicts output
3. Use predictions as new features
4. Train a meta-model on these predictions
5. Meta-model produces final prediction

Why Does Ensemble Learning Work?

Ensemble learning combines multiple classifiers (models) to make a final decision that is more accurate, stable, and generalizable than any individual classifier.

BIAS — SYSTEMATIC MODELING ERROR

Example: A linear SVM assumes that the decision boundary is linear. If the real boundary is nonlinear, the classifier will consistently misclassify points near the true boundary.
Effect: High bias → the model is too simple → underfitting → consistent errors even with large training data.

Why Does Ensemble Learning Work?

VARIANCE — MODEL INSTABILITY

Example: Two decision trees trained on slightly different samples (trees A and B) may produce different split points and very different boundaries.
Effect: High variance → the model is too sensitive to training data → overfitting → inconsistent predictions.

NOISE — DATA IRREGULARITIES

Example: Mislabelled training examples or measurement errors in data collection.
Effect: Noise cannot be reduced by the model—it sets a lower bound on achievable accuracy.

How Ensemble Learning Helps

Ensemble methods—like bagging, boosting, and stacking—improve model performance by reducing bias, variance, or both.

(a) Reducing Bias

- Different models may have different directions of bias in various regions of the input space.
- Combining their predictions (e.g., through majority voting or averaging) can cancel out individual biases, producing a decision boundary that is closer to the true one.

Example:

- Several linear SVMs with different parameter settings each produce slightly different decision boundaries.
- When combined via a majority vote, the resulting ensemble boundary becomes nonlinear and more accurate—closer to the real separation between classes.

(b) Reducing Variance

- Variance comes from sensitivity to the specific training data.
- By training multiple models on different subsets or bootstrap samples, ensemble methods (like Bagging and Random Forests) average out the random fluctuations in each model.
- The ensemble becomes more stable and less likely to overfit to noise.

How Ensemble Learning Helps

c) Balanced Effect (Bias + Variance)

- Some ensemble methods, such as Boosting, explicitly aim to reduce both bias and variance.
- They sequentially train models so that each new model focuses on the previous model's errors, creating a powerful combined classifier.

Gradient Boosting Machine (GBM): Core Idea

Imagine prediction as approximation of a function: y=F(x)
We do not know the function. So we start with a simple guess: $F_0(x) = \text{mean}(y)$

Then we iteratively improve it.
At step m:

$$F_m(x) = F_{m-1}(x) + h_m(x)$$

Where:

$h_m(x)$ = small decision tree that learns current mistakes

So each tree learns: what previous trees failed to learn

Gradient Boosting

$$h_m(x) \approx -\frac{\partial L(y, F(x))}{\partial F(x)}$$

The model minimizes loss using gradient descent in function space.
Meaning: Each new tree predicts the negative gradient = error direction
So boosting = gradient descent but instead of changing weights, we add new trees.

Thank you